

Dominator

Algorithm Design & Analysis Report

Non-Recursive Brute Force | Recursive Divide and Conquer

An array A consisting of N integers is given. The dominator of array A is the value that occurs in more than half of the elements of A.

Given an array A consisting of N integers, return the index of any element of array A in which the dominator of A occurs. The function should return -1 if array A does not have a dominator.

Constraints:

- N is an integer within the range [0..100,000]
- Each element of A is an integer within [-2,147,483,648 .. 2,147,483,647]

Example:

```
A[0]=3  A[1]=4  A[2]=3  A[3]=2  A[4]=3  A[5]=-1  A[6]=3  A[7]=3
Dominator = 3  (appears 5 times out of 8 elements,  $5 > 8/2 = 4$ )
Output: any of indices 0, 2, 4, 6, 7
```

First Algorithm — Non-Recursive Brute Force

1.1 Pseudo Code

The brute-force approach iterates through every element and, for each element, counts how many times its value appears in the entire array. If the count exceeds $N/2$, that element is the dominator.

FUNCTION iterative1Bruteforce(A, N):
FOR i = 0 TO N-1:
count = 0
FOR j = 0 TO N-1:
IF A[j] == A[i]:
count = count + 1
IF count > N / 2:
RETURN i // return index of first dominator occurrence
RETURN -1 // no dominator found
FUNCTION main():
READ n
DECLARE arr[n]
FOR i = 0 TO n-1:
READ arr[i]
index = iterative1Bruteforce(arr, n)
IF index == -1:
PRINT '-1'
ELSE:
value = arr[index]
FOR i = 0 TO n-1:
IF arr[i] == value:
PRINT i // print ALL indices where dominator appears

1.3 Analysis & Complexity

Step-by-step cost analysis of iterative1Bruteforce:

Step	Operation	Cost	Times
1	Outer loop: iterate index i from 0 to N-1	$O(1)$	N
2	Initialize count = 0 (per outer iteration)	$O(1)$	N
3	Inner loop: iterate j from 0 to N-1	$O(1)$	N per i → N^2
4	Compare A[j] == A[i] and increment count	$O(1)$	N^2 total
5	Check if count > N/2 (per outer iteration)	$O(1)$	N

6	Return index if dominator found, else -1	$O(1)$	1
---	--	--------	---

Time Complexity	$O(N^2)$ — nested loops: for each of N elements, scan all N elements
------------------------	--

Space Complexity	$O(1)$ — only a few integer variables (i , j , $count$); no extra data structures
-------------------------	---

Best case: $O(N^2)$ — all comparisons must still be done even if first element is the dominator.

Worst case: $O(N^2)$ — no dominator; all N^2 comparisons are exhausted.

Second Algorithm — Recursive Divide and Conquer

2.1 Pseudo Code

The divide-and-conquer approach splits the array in half recursively. Each half returns a candidate (index) and its count. At merge, if both halves agree on the same value, their counts are summed; otherwise each candidate is verified across the full subarray. The algorithm returns the dominator if any subarray's candidate exceeds half the subarray size.

FUNCTION countOccurrencesRange(A, candidate, start, end):
count = 0
FOR i = start TO end:
IF A[i] == candidate: count = count + 1
RETURN count
FUNCTION getDominatorRecursiveUtil(A, start, end) -> Pair(index, count):
// Base case: single element is always dominator of itself
IF start == end:
RETURN Pair(start, 1)
mid = start + (end - start) / 2
leftResult = getDominatorRecursiveUtil(A, start, mid)
rightResult = getDominatorRecursiveUtil(A, mid+1, end)
// If both halves agree on the same value, merge counts
IF leftResult.index != -1 AND rightResult.index != -1
AND A[leftResult.index] == A[rightResult.index]:
RETURN Pair(leftResult.index,
leftResult.count + rightResult.count)
// Otherwise verify each candidate over the whole subarray
size = end - start + 1
leftCount = 0
rightCount = 0
IF leftResult.index != -1:
leftCount = countOccurrencesRange(A, A[leftResult.index],
start, end)
IF rightResult.index != -1:
rightCount = countOccurrencesRange(A, A[rightResult.index],
start, end)
IF leftCount > size / 2: RETURN Pair(leftResult.index, leftCount)
IF rightCount > size / 2: RETURN Pair(rightResult.index, rightCount)
RETURN Pair(-1, 0) // no dominator in this subarray
FUNCTION recursive6DivideAndConquer(A, N):
IF N == 0: RETURN -1

result = getDominatorRecursiveUtil(A, 0, N-1)
RETURN result.index
FUNCTION main():
READ n
DECLARE arr[n]
FOR i = 0 TO n-1: READ arr[i]
index = recursive6DivideAndConquer(arr, n)
IF index == -1:
PRINT 'Result: -1'
ELSE:
value = arr[index]
FOR i = 0 TO n-1:
IF arr[i] == value: PRINT i

2.3 Analysis & Complexity

Let $T(N)$ be the time to solve a subarray of size N . The recurrence relation is:

$T(N) = 2 * T(N/2) + O(N)$
^^^^^^^^^ ^^^
two recursive countOccurrencesRange scans the
sub-calls whole subarray in $O(N)$ at merge step

Applying the Master Theorem (Case 2: $a=2$, $b=2$, $f(N)=O(N)$, $\log_b(a)=1 = \text{degree of } f$):

Level	What Happens	Work	Calls
0 (top)	Split array, countOccurrencesRange over N elements	$O(N)$	1
1	Two halves $N/2$, scan $N/2$ each	$O(N/2)$	2
2	Four quarters $N/4$, scan $N/4$ each	$O(N/4)$	4
...	...	...	...
$\log N$	Base cases: single elements	$O(1)$	N

Total work per level = $O(N)$ across all calls at every depth. There are $\log N$ levels.

Therefore total work = $O(N) \times \log N = O(N \log N)$.

Time Complexity	$O(N \log N)$ — by Master Theorem: $T(N) = 2T(N/2) + O(N)$
------------------------	--

**Space
Complexity**

$O(\log N)$ – recursion call stack depth is $\log N$
(binary split each level)

Comparison of Both Algorithms

Aspect	Algorithm 1 — Brute Force (Non-Recursive)	Algorithm 2 — Divide & Conquer (Recursive)
Approach	Nested iteration over all pairs	Recursive binary split + merge
Time Complexity	$O(N^2)$ — quadratic	$O(N \log N)$ — linearithmic
Space Complexity	$O(1)$ — no extra memory	$O(\log N)$ — recursion stack
Best Case	$O(N^2)$ — no early exit in inner loop	$O(N \log N)$ — always splits fully
Worst Case	$O(N^2)$ — no dominator exists	$O(N \log N)$ — no dominator exists
Code Simplicity	Very simple — two nested loops	More complex — struct, recursion
Scalability	Poor — slow for large N	Good — handles N=100,000 well
N = 100,000 (approx ops)	~10,000,000,000 (10 billion)	~1,700,000 (1.7 million)
Stack Overflow Risk	None — fully iterative	Low risk — only log N deep
Best Use Case	Small arrays, quick prototyping	Large arrays, production use

Conclusion: Algorithm 2 (Divide and Conquer) is significantly more efficient for large inputs. At N=100,000 the brute-force performs roughly 10 billion operations while the divide-and-conquer approach needs only about 1.7 million — a speedup of nearly 5,900x. Algorithm 1 is easier to understand and suitable for small inputs or educational purposes.